# Abalony

-Showcase2- Group 7

**ST271**

# Project Contents

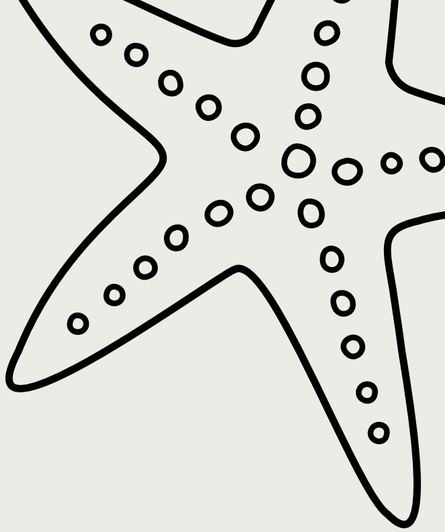

01 Dataset

02 Methodology

03 Experimental Plan

04 Discuss Results

05 Code Description



# Dataset

1. **Id** : เลขประจำตัวของข้อมูลในแต่ละแถว
2. **Sex** : เพศ
3. **Length** :  ความยาว
4. **Diameter** : เส้นผ่านศูนย์กลาง
5. **Height** : ความสูง
6. **Whole weight** : น้ำหนักทั้งหมด
7. **Whole weight.1** : น้ำหนักส่วนที่ 1
8. **Whole weight.2** :  น้ำหนักส่วนที่ 2
9. **Shell weight** : น้ำหนักเปลือก
10. **Rings** : จำนวนวงบนเปลือกหอย

(29192,9)



# Dataset

Id : Numerical
**Sex : Categorical**
Length : Numerical
Diameter : Numerical
Height : Numerical

Whole weight : Numerical
Whole weight.1 : Numerical
Whole weight.2 : Numerical
Shell weight : Numerical
Rings : Numerical

| id | Sex | Length | Diameter | Height | Whole weight | Whole weight.1 | Whole weight.2 | Shell weight | Rings |
|---|---|---|---|---|---|---|---|---|---|
| **0** | F | 0.550 | 0.430 | 0.150 | 0.7715 | 0.3285 | 0.1465 | 0.2400 | 11 |
| **1** | F | 0.630 | 0.490 | 0.145 | 1.1300 | 0.4580 | 0.2765 | 0.3200 | 11 |
| **2** | I | 0.160 | 0.110 | 0.025 | 0.0210 | 0.0055 | 0.0030 | 0.0050 | 6 |
| **3** | M | 0.595 | 0.475 | 0.150 | 0.9145 | 0.3755 | 0.2055 | 0.2500 | 10 |
| **4** | I | 0.555 | 0.425 | 0.130 | 0.7820 | 0.3695 | 0.1600 | 0.1975 | 9 |

(29192,9)

# Cleaning Dataset

1

ตรวจสอบ Missing values และ Duplicate data

```
data.isnull().sum()
```

```
Sex               0
Length            0
Diameter          0
Height            0
Whole weight      0
Whole weight.1    0
Whole weight.2    0
Shell weight      0
Rings             0
dtype: int64
```

```
data.duplicated().sum()
```

```
0
```

**Missing values = 0**
**Duplicate datas = 0**

# Cleaning Dataset

2

Encoding ข้อมูลที่เป็น Categorical data

ด้วย One Hot Encoding

| id | Length | Diameter | Height | Whole weight | Whole weight.1 | Whole weight.2 | Shell weight | F | I | M | Rings |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | Sex | | | |
| 0 | 0.550 | 0.430 | 0.150 | 0.7715 | 0.3285 | 0.1465 | 0.2400 | 1 | 0 | 0 | 11 |
| 1 | 0.630 | 0.490 | 0.145 | 1.1300 | 0.4580 | 0.2765 | 0.3200 | 1 | 0 | 0 | 11 |
| 2 | 0.160 | 0.110 | 0.025 | 0.0210 | 0.0055 | 0.0030 | 0.0050 | 0 | 1 | 0 | 6 |
| 3 | 0.595 | 0.475 | 0.150 | 0.9145 | 0.3755 | 0.2055 | 0.2500 | 0 | 0 | 1 | 10 |
| 4 | 0.555 | 0.425 | 0.130 | 0.7820 | 0.3695 | 0.1600 | 0.1975 | 0 | 1 | 0 | 9 |

1 หมวด = 1 feature

1 คือ มีลักษณะนั้น, 0 คือ ไม่มีลักษณะนั้น

# Cleaning Dataset

3

Scaling ทุก feature ให้อยู่ใน range เดียวกัน

**ด้วย Standardization**

```
# Scaling
scaler = preprocessing.StandardScaler() # mean = 0 , std = 1
scaler.fit(X_train)
X_tr = scaler.transform(X_train)
X_te = scaler.transform(X_test)
```

**โดยแปลง raw data ให้ค่า mean = 0 และ std = 1**

# Methodology

วิธีที่ใช้ในการศึกษาครั้งนี้คือ **Deep Learning**
โดยใช้โครงสร้าง Neural Network ที่ประกอบด้วยหลาย layers
เพื่อเรียนรู้ความสัมพันธ์ที่ซับซ้อนจากข้อมูล

แต่ละ layer ประกอบด้วย neurons ที่เชื่อมต่อกันเป็นลำดับชั้น
แต่ละชั้นจะรับข้อมูลจากชั้นก่อนหน้า ทำการคำนวณ และส่งผลลัพธ์ไปยังชั้นถัดไป
กระบวนการเรียนรู้เกิดขึ้นจากการปรับค่า weights ผ่านขั้นตอนการ train
เพื่อให้ค่า loss  ลดลงอย่างต่อเนื่อง และเพิ่มความแม่นยำในการทำนาย Rings

# Experimental Plan

## Model 1

INPUT LAYER: 10 neurons
HIDDEN LAYER: 3 layers
1. 16 neurons
2. 8 neurons
3. 4 neurons
OUTPUT LAYER: 1

**รวม 28**

## Model 2

INPUT LAYER: 10 neurons
HIDDEN LAYER: 3 layers
1. 32 neurons
2. 16 neurons
3. 8 neurons
OUTPUT LAYER: 1

**รวม 56**

## Model 3

INPUT LAYER: 10 neurons
HIDDEN LAYER: 3 layers
1. 64 neurons
2. 32 neurons
3. 16 neurons
OUTPUT LAYER: 1

**รวม 112**

## Model 4

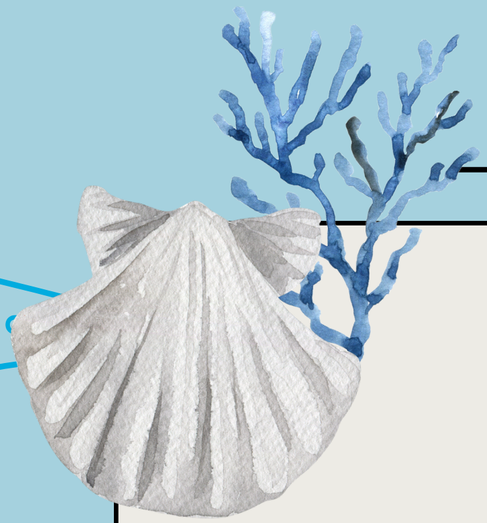
INPUT LAYER: 10 neurons
HIDDEN LAYER: 3 layers
1. 128 neurons
2. 64 neurons
3. 32 neurons
OUTPUT LAYER: 1

**รวม 224**

## Objective

ศึกษาและเปรียบเทียบผลกระทบของ
**จำนวน neuron** ต่อ **ประสิทธิภาพ** ในการทำนายของโมเดล
ภายใต้โครงสร้างที่มีจำนวน Hidden Layers เท่ากัน

# Hypothesis



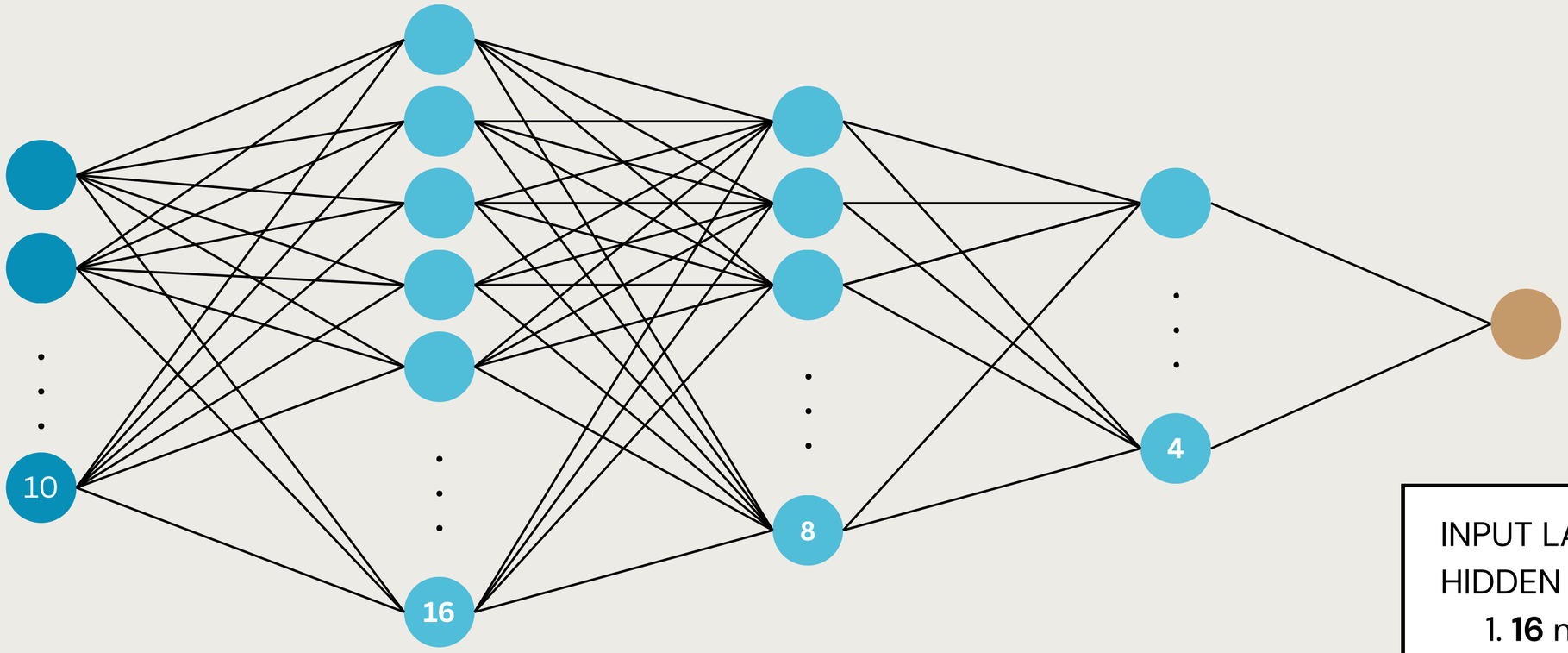
โมเดลที่มีจำนวน neurons มากที่สุด
จะมีประสิทธิภาพในการทำนายดีที่สุด



28 neurons

# Model 1

input layer | hidden layer1 | hidden layer2 | hidden layer3 | output layer

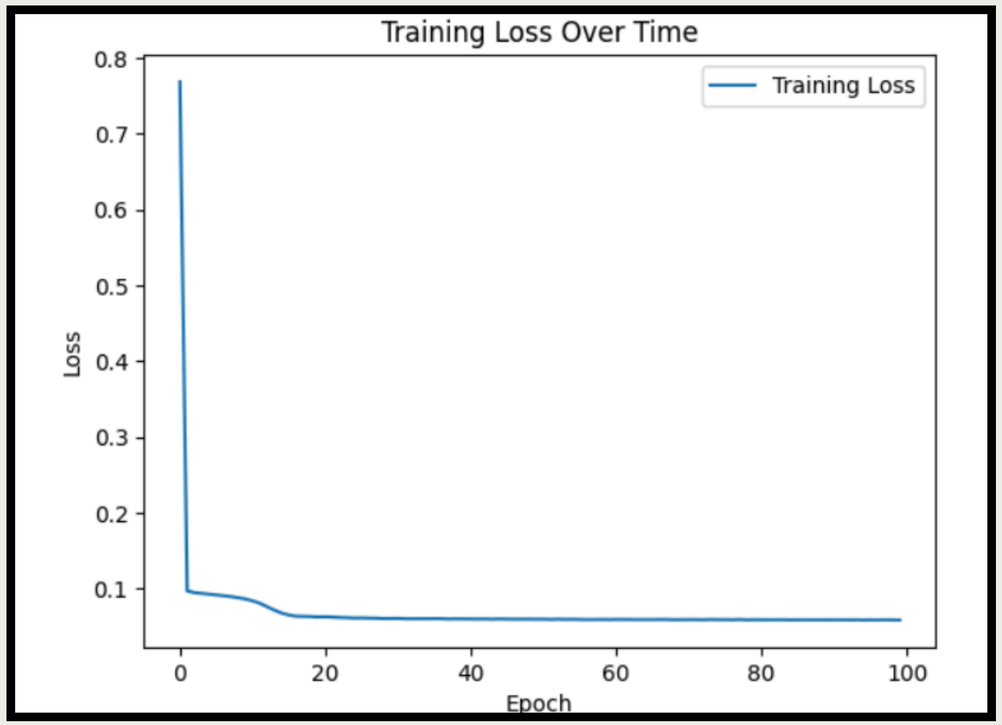


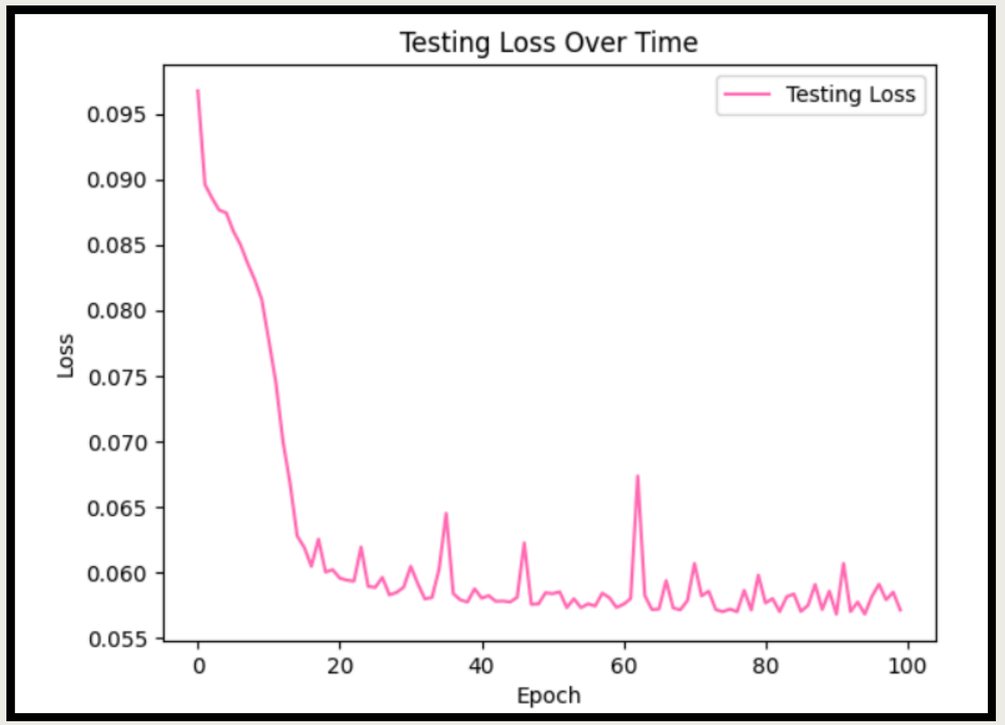

INPUT LAYER: 10 neurons
HIDDEN LAYER: 3 layers
1. **16** neurons
2. **8** neurons
3. **4** neurons
OUTPUT LAYER: 1

28 neurons

# Discuss Results

## Model 1



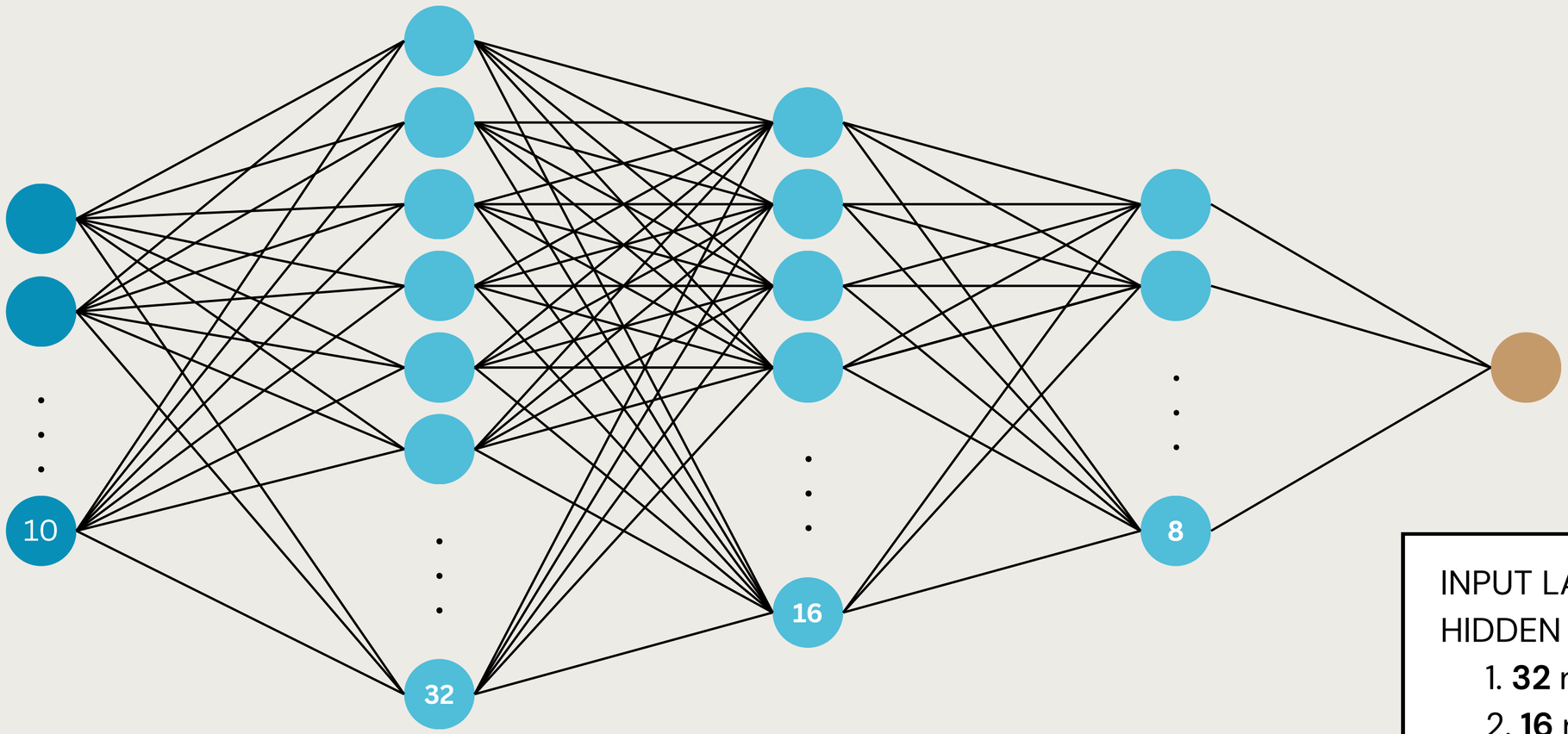




```
Training Performance: 0.0001588817511167188 6
Testing Performance: 0.0001579309762511953 6
```

**56 neurons**

# Model 2

input layer | hidden layer1 | hidden layer2 | hidden layer3 | output layer

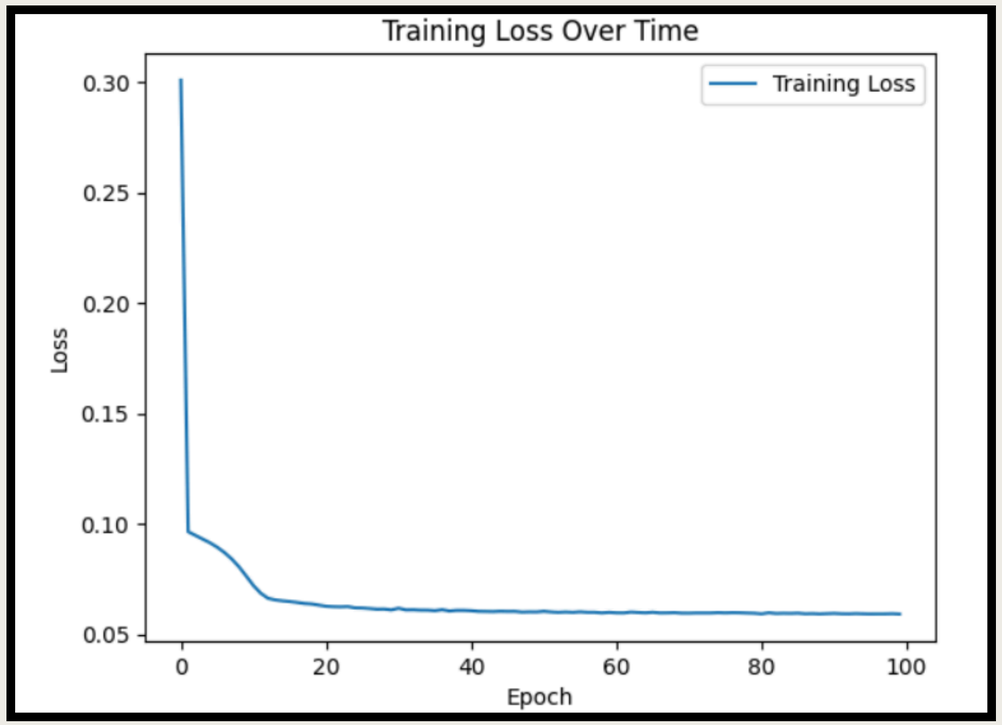


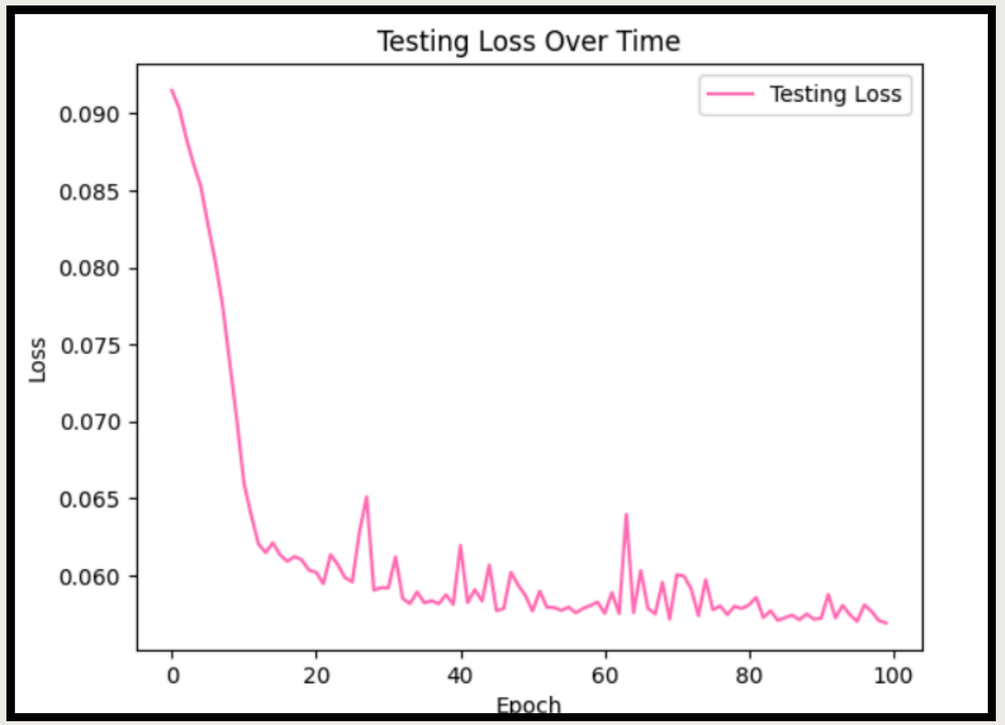

INPUT LAYER: 10 neurons
HIDDEN LAYER: 3 layers
1. **32** neurons
2. **16** neurons
3. **8** neurons
OUTPUT LAYER: 1

56 neurons

# Discuss Results

## Model 2

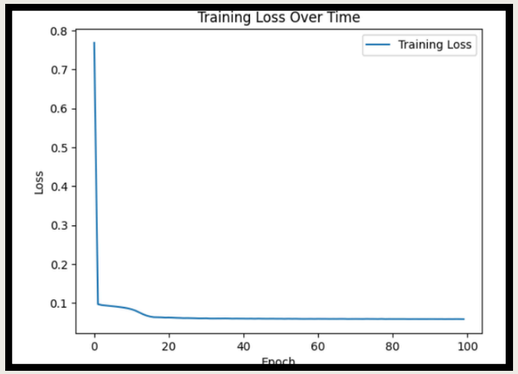



```
Training Performance: 0.00015938072211123816
Testing Performance: 0.00015820397384487393
```

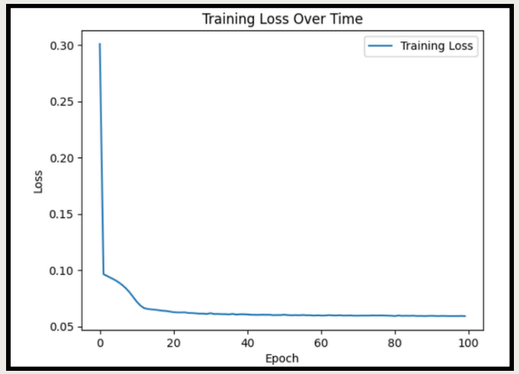
# Discuss Results

**Model 1**




**0.00015793097625119536**

Model 2



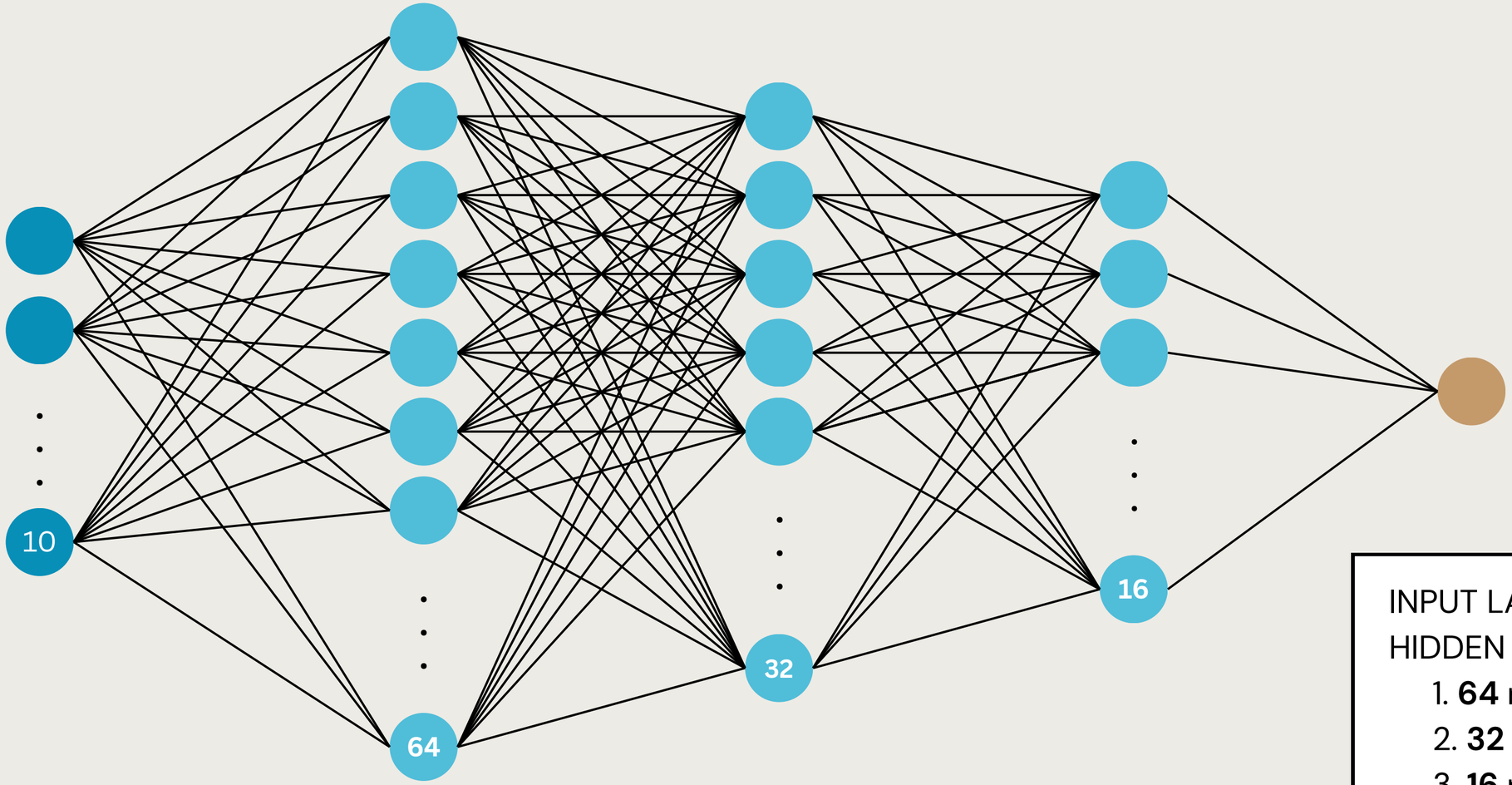

0.00015820397384487393

**Best Performance** → **Worst Performance**

**112 neurons**

# Model 3

input layer | hidden layer1 | hidden layer2 | hidden layer3 | output layer

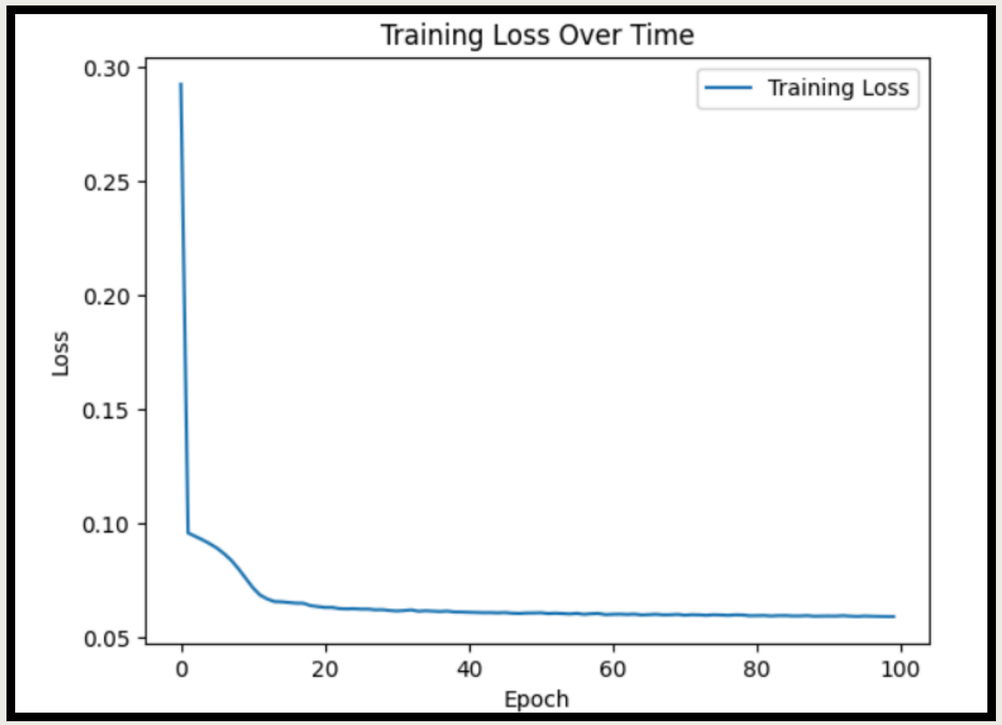


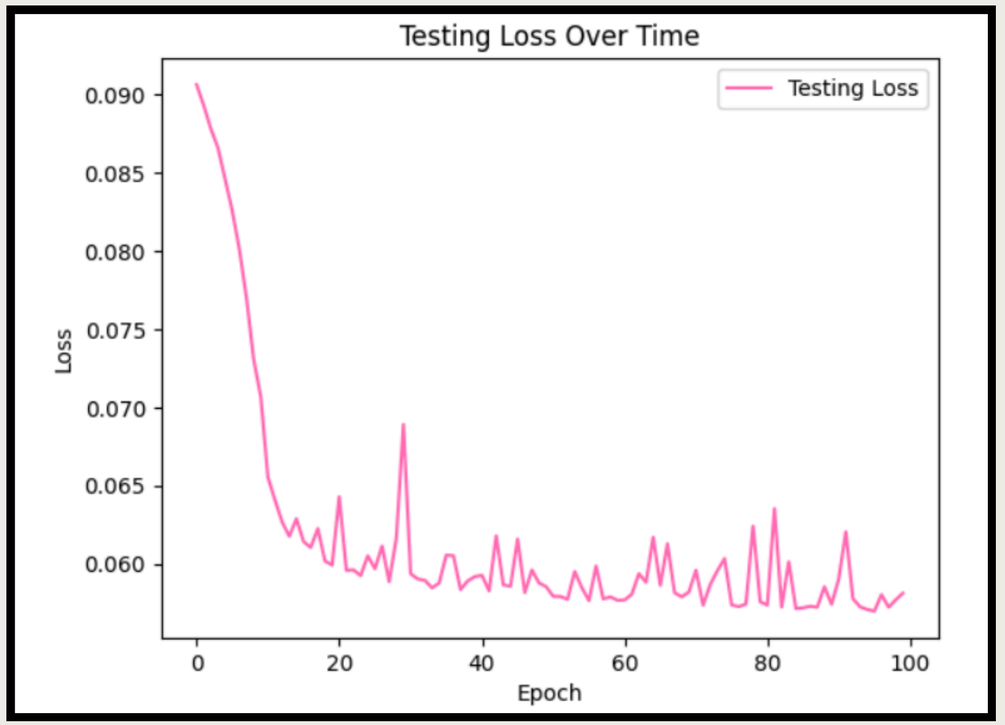

INPUT LAYER: 10 neurons
HIDDEN LAYER: 3 layers
1. **64** neurons
2. **32** neurons
3. **16** neurons
OUTPUT LAYER: 1

112 neurons

# Discuss Results

## Model 3

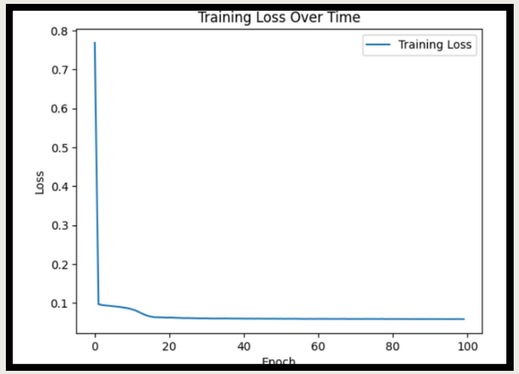



```
Training Performance: 0.0001608870174728287
Testing Performance: 0.00016006984195326174
```

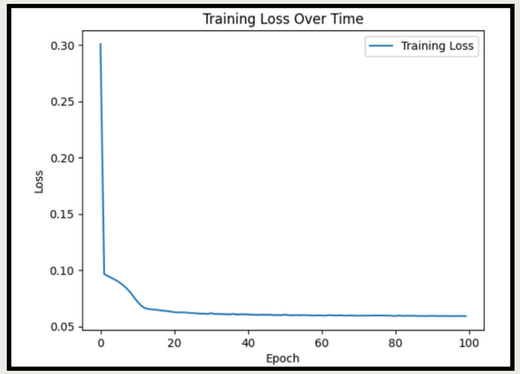
# Discuss Results

**Model 1**



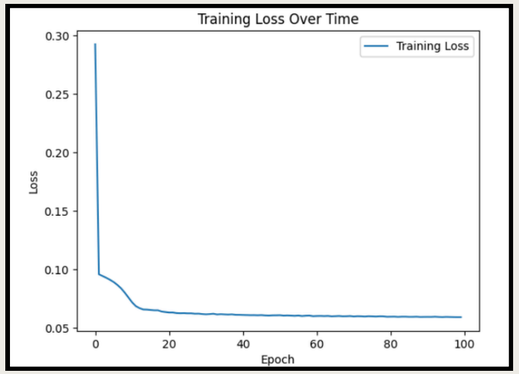

**0.00015793097625119536**

Model 2




0.00015820397384487393

Model 3



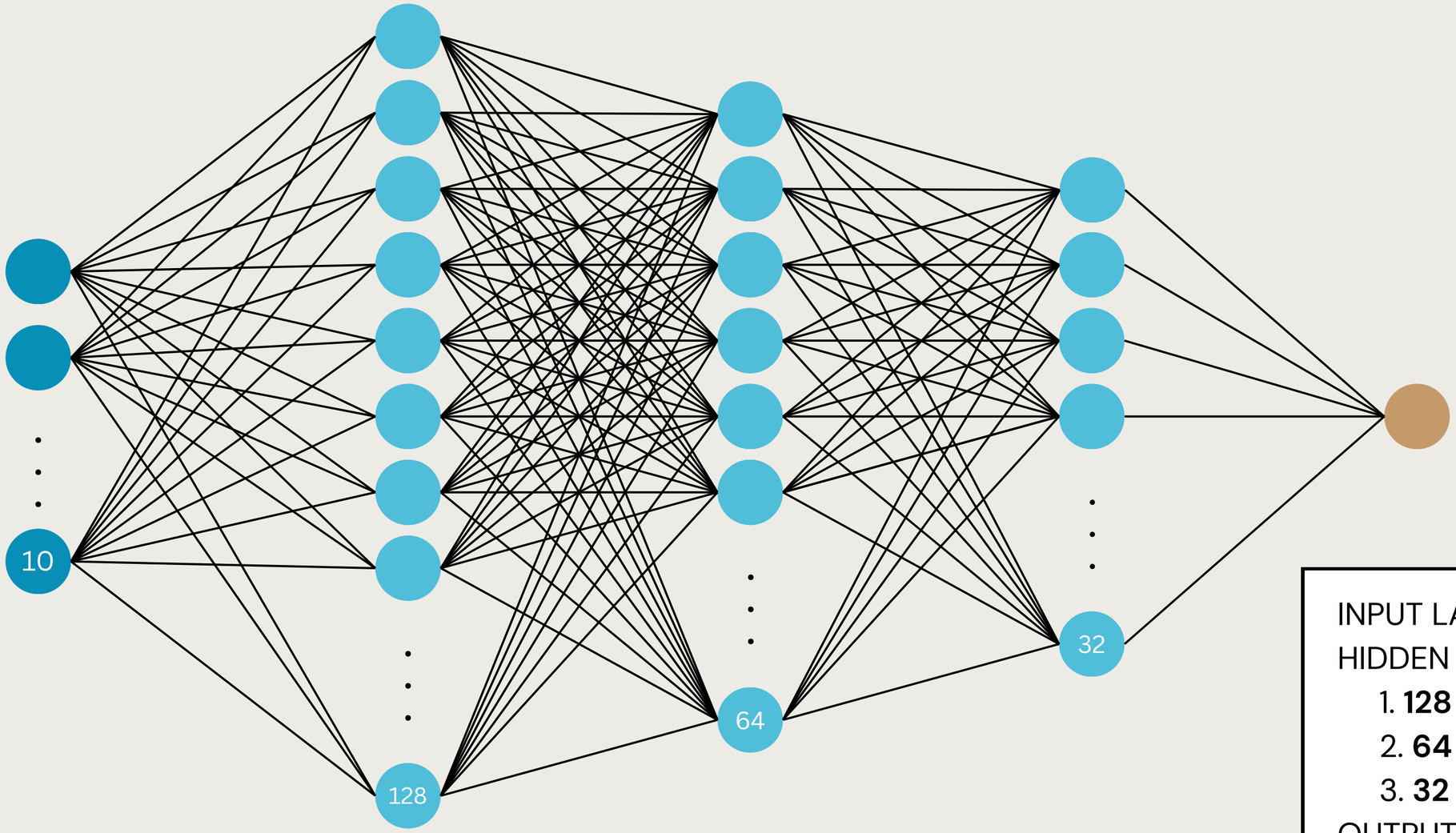

0.00016006984195326174

**Best Performance** → **Worst Performance**

224 neurons

# Model 4

input layer | hidden layer1 | hidden layer2 | hidden layer3 | output layer

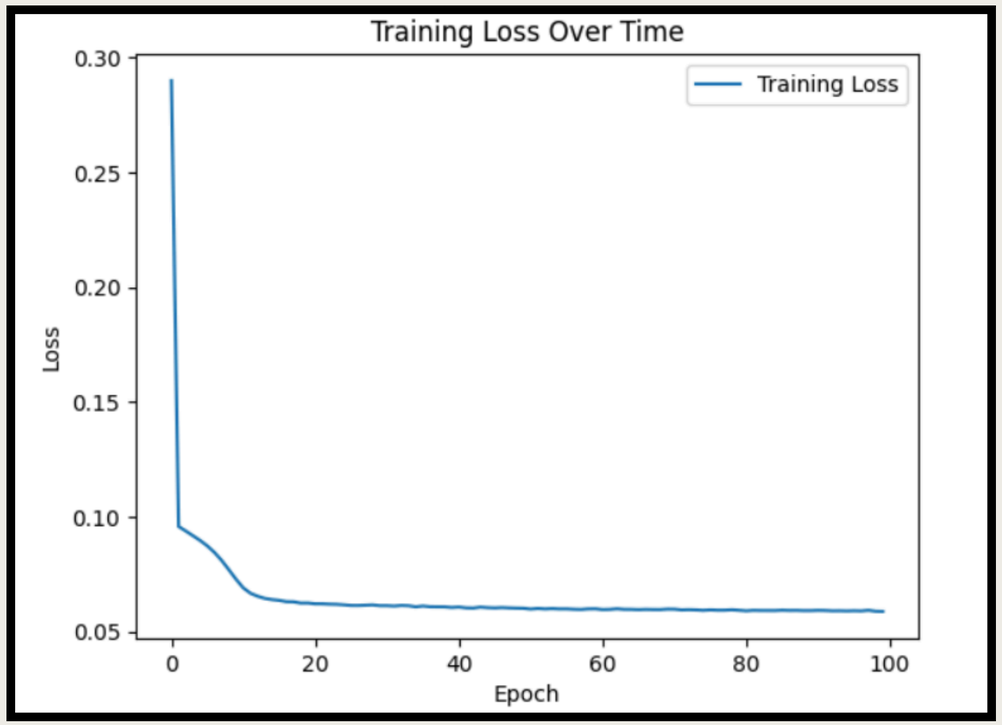


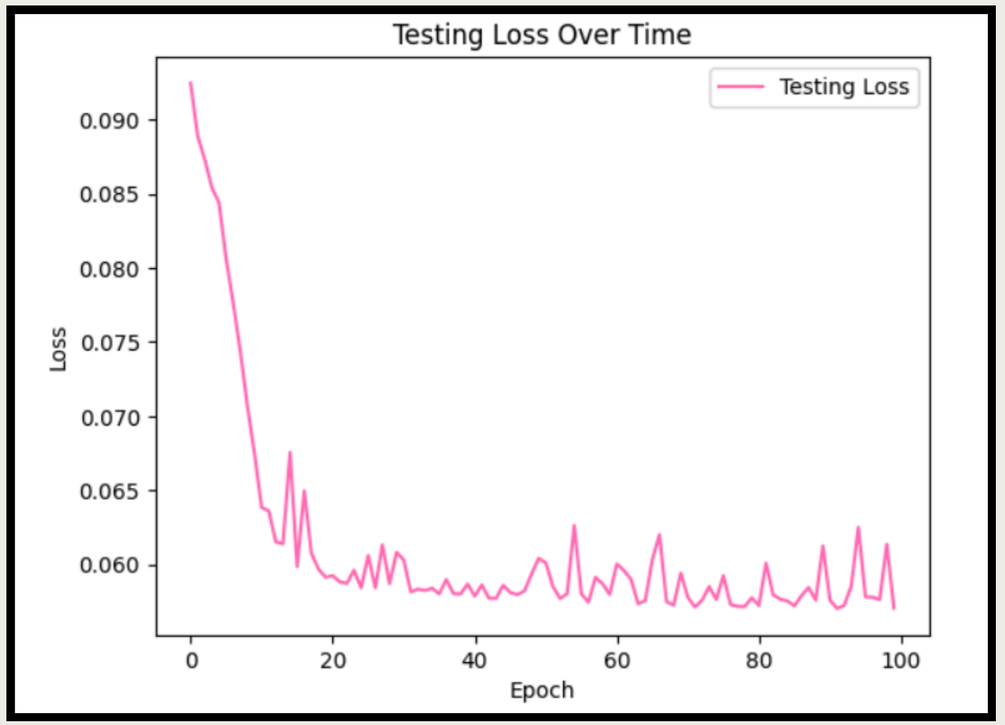

INPUT LAYER: 10 neurons
HIDDEN LAYER: 3 layers
1. **128** neurons
2. **64** neurons
3. **32** neurons
OUTPUT LAYER: 1

224 neurons

# Discuss Results

## Model 4

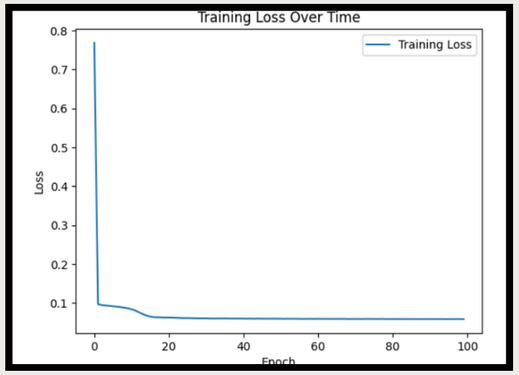



```
Training Performance: 0.00016052821042608848
Testing Performance: 0.00015923189043881312
```

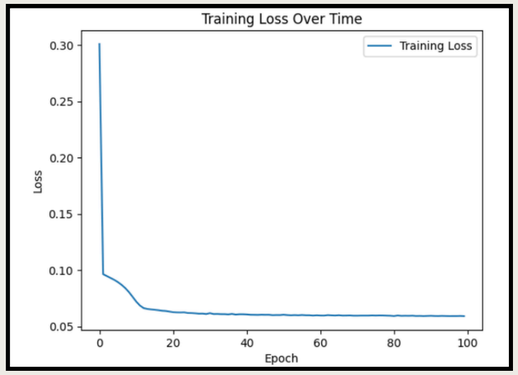
# Discuss Results

**Model 1**



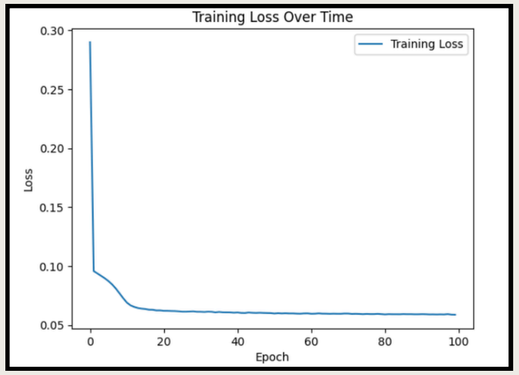

**0.00015793097625119536**

Model 2



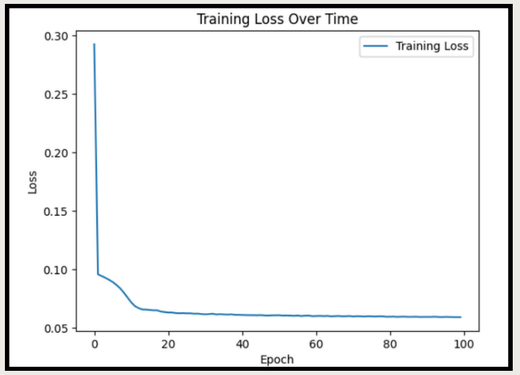

0.00015820397384487393

Model 4




0.00015923189043881312

Model 3



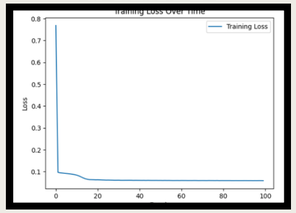

0.00016006984195326174

**Best Performance**

**Worst Performance**

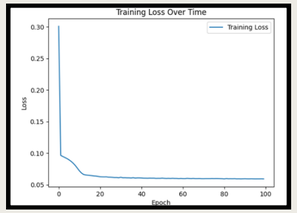
# Discuss Results

**Model 1**



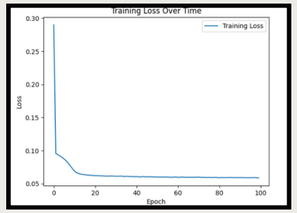

**0.00015793097625119536**

Model 2



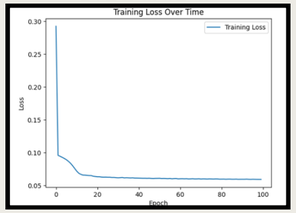

0.00015820397384487393

Model 4




0.00015923189043881312

Model 3

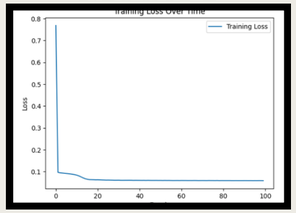



0.00016006984195326174

จากผลการทดลองพบว่า โมเดลที่มีจำนวน neurons น้อยที่สุด ซึ่งก็คือ Model 1 มีประสิทธิภาพในการทำนายสูงที่สุด
ซึ่งขัดแย้งกับสมมติฐานที่ตั้งไว้ว่าโมเดลที่มีจำนวน neurons มากที่สุด จะมีประสิทธิภาพในการทำนายดีที่สุด

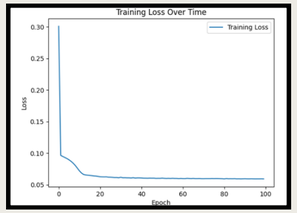
# Discuss Results

**Model 1**



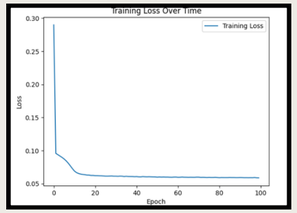

**0.00015793097625119536**

Model 2



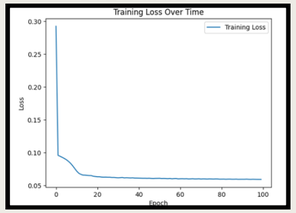

0.00015820397384487393

Model 4




0.00015923189043881312

Model 3




0.00016006984195326174

ผลลัพธ์นี้อาจอธิบายได้ว่า ชุดข้อมูล Abalony ไม่ได้มีความซับซ้อนสูงมาก การเพิ่มจำนวน neurons มากเกินไปไม่ได้ช่วยให้โมเดลเรียนรู้ได้ดีขึ้น และยังอาจทำให้เสี่ยงต่อการเกิด Overfitting ได้มากขึ้น

ดังนั้น สำหรับชุดข้อมูลนี้ โมเดลที่มี neurons น้อยจึงมีความเหมาะสมมากกว่าทั้งในด้านประสิทธิภาพและความคุ้มค่า

# Code Description

| | |
|---|---|
| pandas | ใช้อ่านไฟล์และจัดการข้อมูลในรูปแบบ Data frame |
| numpy | ใช้คำนวณทางคณิตศาสตร์และจัดการชุดข้อมูล |
| sklearn | ใช้แบ่งข้อมูลและเตรียมข้อมูลก่อนนำไปฝึกโมเดล |
| pytorch | ใช้สร้างโครงสร้าง model และฝึก model |
| matplotlib | ใช้วาดกราฟแสดงการเปลี่ยนแปลงค่า loss ตาม จำนวน epochs |

# pandas

**read_csv()**: อ่านไฟล์ CSV เข้ามาเป็น DataFrame
**get_dummies()**: ทำ One-Hot Encoding เปลี่ยนข้อมูล Categorical เป็นตัวเลข
**concat()**: นำ DataFrame หลายอันมาเชื่อมต่อกัน

# sklearn

**train_test_split**: สุ่มแบ่งข้อมูลเป็นชุด Train และชุด Test

# pytorch

**tensor()**: แปลง array เป็น Tensor เพื่อใช้ประมวลผลใน model

# numpy

**array()**: แปลงข้อมูลจาก List หรือ DataFrame ให้เป็น array

# matplotlib

**plot()**: วาดกราฟเส้นแสดงแนวโน้มของค่า Loss
**title(), xlabel(), ylabel()**: กำหนดชื่อกราฟและชื่อแกน X, Y
**legend()**: แสดงกล่องอธิบายเส้นกราฟ (Label)

# Members

1. นางสาวนาขวัญ จิตต์เอื้อเฟื้อ 67102010400
2. นางสาวปุณณภา จึงเปี่ยมสุข 67102010401
3. นางสาวอุษณา อุตสาหะ 67102010411